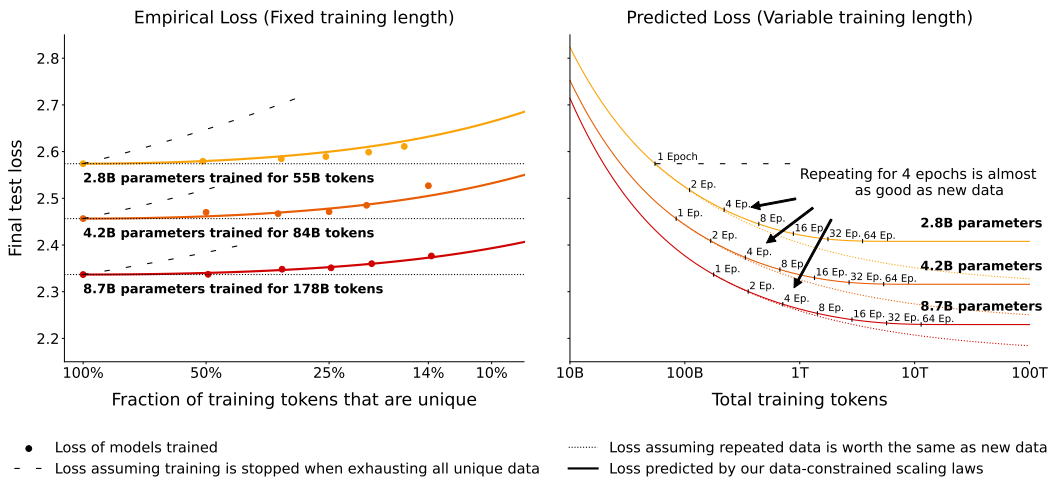

Figure 5: **Empirical and Extrapolated loss with constrained data.** *(Left):* Loss as a function of repeated tokens for three different training budgets each with fixed number of parameters. Loss curves predicted by our data-constrained scaling laws are shifted to exactly match the loss at 100% unique data. Return on FLOPs decays with repeated data in a regular pattern. *(Right):* Extrapolating from the proposed data-constrained scaling law shows that at small numbers epochs are benign, but at large number of epochs loss stops improving.

Adding parameters and epochs causes the loss to decrease and eventually increase again, suggesting that too much compute can hurt performance. Results from [46] also show that loss can increase when too many parameters are used, even with early stopping. However, we expect that appropriate regularization (such as simply removing all excess parameters as an extreme case) could prevent this behavior. Thus, our formula presented in §3 and its predicted isoLoss contours in Figure 3 do not model the possibility that excess epochs or parameters could hurt performance.

6 Results: Resource Return for Data-Constrained Scaling

Next, consider the question of *Return* on scaling. To quantify this value, we run experiments with three FLOP budgets across eight respective data budgets to compare return on FLOPs.

Figure 4 shows the configurations and validation curves for models trained on the same number of total tokens. Conforming to intuition and prior work on deduplication [55], repeated data is worth less, thus models trained on less unique data (and, correspondingly, more epochs) have consistently higher loss. However, the loss difference for a few epochs is negligible. For example, the $N = 8.7$ billion parameter model trained for four epochs ($D_C = 44$ billion unique tokens) finishes training with only 0.5% higher validation loss than the single-epoch model ($D_C = 178$ billion unique tokens).

In Figure 5 (left), we compare the final test loss of each model to predictions from our parametric fit. The data-constrained scaling laws can accurately measure the decay in the value of repeated data as seen by the proximity of empirical results (dots) and parametric fit (lines). We note however that it significantly underestimates the final test loss of failing models where loss increases midway through training, such as models trained for 44 epochs (not depicted).

In Figure 5 (right), we extrapolate the three budgets by further scaling compute while keeping the data constraints (D_C) at 55B, 84B, and 178B tokens, respectively. The parameter R_D^* introduced in §3 represents roughly the "half-life" of epochs: specifically the point where repeated tokens have lost $\frac{1}{e}$ of their value. Through our fitting in Appendix A, we found $R_D^* \approx 15$, corresponding to 15 repetitions (or 16 epochs). Graphically, this can be seen by the stark diminishing returns in the proximity of the 16-epoch marker and the flattening out soon after.

Overall, the *Return* when repeating data is relatively good. Meaningful gains from repeating data can be made up to around 16 epochs (R_D^*) beyond which returns diminish extremely fast.

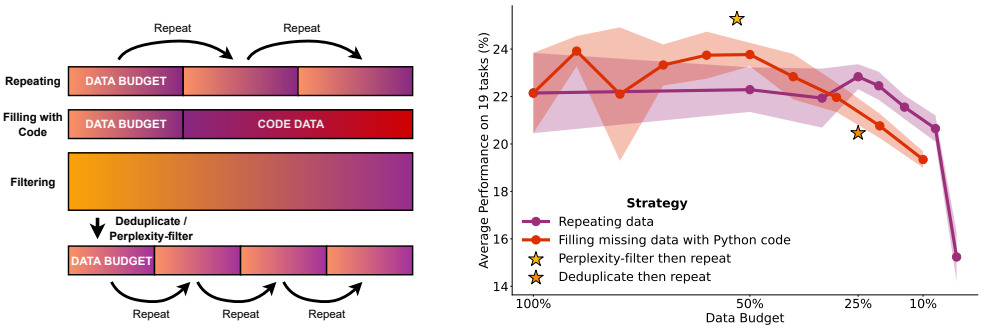

Figure 6: **Strategies for data-constrained settings and their downstream performance.** *(Left):* Schematic showing alternative data use strategies of code filling and filtering. *(Right):* $N = 4.2$ billion parameter models trained for a total of $D = 84$ billion tokens with varying budgets D_C. For repeating and filling with code, five models with different seeds are trained for each dot and the standard deviation is visualized as the shaded area.

7 Results: Complementary Strategies for Obtaining Additional Data

While repeating data is effective, it has diminishing returns. We therefore consider strategies for scaling D targeting improved downstream performance as opposed to directly minimizing loss.

Figure 6 (left) illustrates the strategies: **(a) Code augmentation:** We use Python code from The Stack [49] to make up for missing natural language data. The combined dataset consisting of code and natural language samples is shuffled randomly. **(b) Adapting filtering:** We investigate the performance impact of deduplication and perplexity filtering, two common filtering steps that can severely limit available data. Removing such filtering steps can free up additional training data.

For these experiments, we set a maximum data budget (D_C) of 84 billion tokens. For repetition and code filling, only a subset of D_C is available and the rest needs to be compensated for via repeating or adding code. For both filtering methods, we start out with approximately twice the budget (178 billion tokens), as it is easier to gather noisy data and filter it than it is to gather clean data for training. For perplexity filtering, we select the top 25% samples with the lowest perplexity according to a language model trained on Wikipedia. This results in 44 billion tokens that are repeated for close to two epochs to reach the full data budget. For deduplication filtering, all samples with a 100-char overlap are removed resulting in 21 billion tokens that are repeated for four epochs during training. See Appendix N for more details on the filtering procedures.

When comparing across data strategies, loss ceases to be a good evaluation metric as the models are trained on different data distributions. We thus evaluate models on 19 natural language tasks with zero to five in-context few-shot exemplars [15] producing 114 scores per model. As our evaluation tasks cover different metrics and random baselines, we re-scale all scores to be in the same range to better reflect performance ranges before averaging. Details on the evaluation datasets are in Appendix K.

In Figure 6 (right) we compare the downstream performance of all strategies. For repeating data, differences in downstream performance are insignificant for up to around 4 epochs (25% budget) and then start dropping, which aligns with our results on test loss in §6. Filling up to 50% of data with code (42 billion tokens) also shows no deterioration. Beyond that, performance decreases quickly on natural language tasks. However, adding more code data may benefit non-natural language tasks, which are not considered in the benchmarking. Two of the tasks benchmarked, WebNLG [17, 34], a generation task, and bAbI [122, 57], a reasoning task, see jumps in performance as soon as code is added, possibly due to code enabling models to learn long-range state-tracking capabilities beneficial for these tasks.

Of the filtering approaches, we find perplexity-filtering to be effective, while deduplication does not help. Prior work found deduplication was able to improve perplexity [55]; however, it did not evaluate on downstream tasks. Deduplication may have value not captured in our benchmark, such as reducing memorization [45, 40, 16, 10]. We also investigate filtering on a different noisier dataset in Appendix O, where we find it to be more effective. Overall, in a data-constrained regime, we recommend reserving filtering for noisy datasets and using both code augmentation and repeating to

increase data tokens. For example, first doubling the available data by adding code and then repeating the new dataset for four epochs results in 8× more training tokens that are expected to be just as good as having had 8× more unique data from the start.

8 Related Work

Large language models Scaling up transformer language models [111] across parameter count and training data has been shown to result in continuous performance gains [19]. Starting with the 1.4 billion parameter GPT-2 model [88], a variety of scaled-up language models have been trained, commonly referred to as large language models (LLMs). They can be grouped into dense models [15, 47, 58, 89, 20, 13, 132, 109, 103, 108, 130, 95, 56, 65] and sparse models [30, 131, 28, 135] depending on whether each forward pass makes use of all parameters. These models are generally pre-trained to predict the next token in a sequence, which makes them applicable to various language tasks directly after pre-training [15, 118, 50, 71, 102] by reformulating said NLP tasks as context continuation tasks (see [67] for an earlier proposal on this topic). We focus on the most common scenario, where a dense transformer model is trained to do next-token prediction on a large corpus and evaluated directly after pre-training using held-out loss or zero- to few-shot prompting.

Scaling laws Prior work has estimated an optimal allocation of compute for the training of LLMs. Kaplan et al. [46] suggested a 10× increase in compute should be allocated to a 5.5× increase in model size and a 1.8× increase in training tokens. This first scaling law has led to the creation of very large models trained on relatively little data, such as the 530 billion parameter MT-NLG model trained on 270 billion tokens [99]. More recent work [42], however, showed that model size and training data should rather be scaled in equal proportions. These findings called for a renewed focus on the scaling of pre-training data rather than scaling model size via complex parallelization strategies [98, 91, 9, 78]. Up-sampling is often employed when pre-training data is partly limited, such as data from a high-quality domain like Wikipedia or text in a rare language for training multilingual LLMs [60, 82]. Hernandez et al. [40] study up-sampling of data subsets and find that repeating only 0.1% of training data 100 times significantly degrades performance. In contrast, our work focuses on repeating the entire pre-training corpus for multiple epochs rather than up-sampling parts of it.

Alternative data strategies Large pre-training datasets are commonly filtered to remove undesired samples or reduce noise [101]. Perplexity-based filtering, whereby a trained model is used to filter out samples with high perplexity, has been found beneficial to reduce noise in web-crawled datasets [121]. Mixing of data is employed for the pre-training data of multilingual LLMs, where text data from different languages is combined [23, 126, 100, 74]. However, both for code and natural language models, mixing different (programming) languages has been reported to under-perform monolingual models [80, 113]. Some work has investigated mixing code and natural language data for prediction tasks, such as summarizing code snippets [44] or predicting function names [4]. Several pre-training datasets for LLMs include low amounts of code data [31, 89, 95]. However, these past works generally do not provide any ablation on the drawbacks of including code or the benefits for natural language task performance. We perform a detailed benchmarking of mixing Python and natural language in LLM pre-training at 10 different mixing rates.

9 Conclusion

This work studies data-constrained scaling, focusing on the optimal use of computational resources when unique data is limited. We propose an extension to the Chinchilla scaling laws that takes into account the decay in value of repeated data, and we fit this function using a large set of controlled experiments. We find that despite recommendations of earlier work, training large language models for multiple epochs by repeating data is beneficial and that scaling laws continue to hold in the multi-epoch regime, albeit with diminishing returns. We also consider complementary approaches to continue scaling models, and find that code gives the ability to scale an additional 2×. We believe that our findings will enable further scaling of language models to unlock new capabilities with current data. However, our work also indicates that there are limits on the scaling horizon. In addition to collecting additional data, researchers should explore using current data in a more effective manner.